PH3120

Computational Physics Laboratory 1

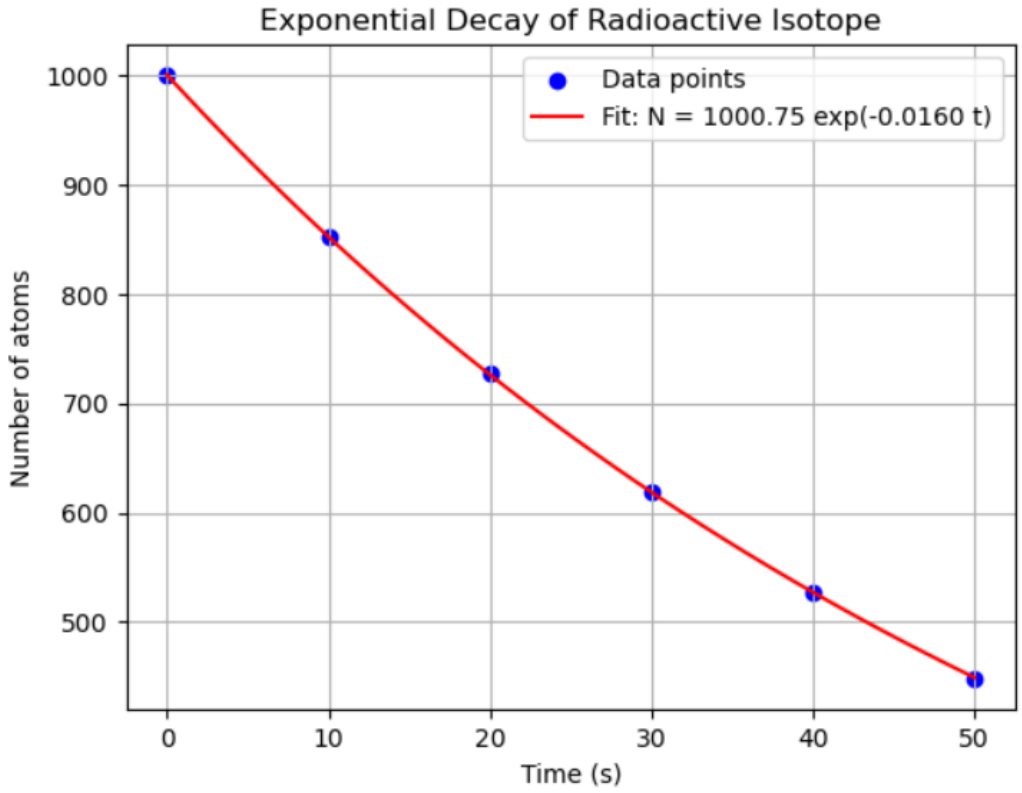
CPL107 – Numerical Methods (Revision) Lab Sheet

1.

(a)

```python
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

t = np.array([0, 10, 20, 30, 40, 50])
N = np.array([1000, 853, 727, 619, 527, 448])

def exp_decay(t, N0, lambda_):
    return N0 * np.exp(-lambda_ * t)

params, covariance = curve_fit(exp_decay, t, N, p0=[1000, 0.02])
N0_fit, lambda_fit = params

print(f"Initial number of atoms (N0): {N0_fit:.2f}")
print(f"Decay constant (λ): {lambda_fit:.4f}")

t_fit = np.linspace(0, 50, 100)
N_fit = exp_decay(t_fit, N0_fit, lambda_fit)

plt.scatter(t, N, color='blue', label='Data points')
plt.plot(t_fit, N_fit, color='red', label=f'Fit: N = {N0_fit:.2f} exp(-{lambda_fit:.4f} t)')
plt.xlabel('Time (s)')
plt.ylabel('Number of atoms')
plt.title('Exponential Decay of Radioactive Isotope')
plt.legend()
plt.grid(True)
plt.show()
```

```
Initial number of atoms (N0): 1000.75
Decay constant (λ): 0.0160
```



(b)

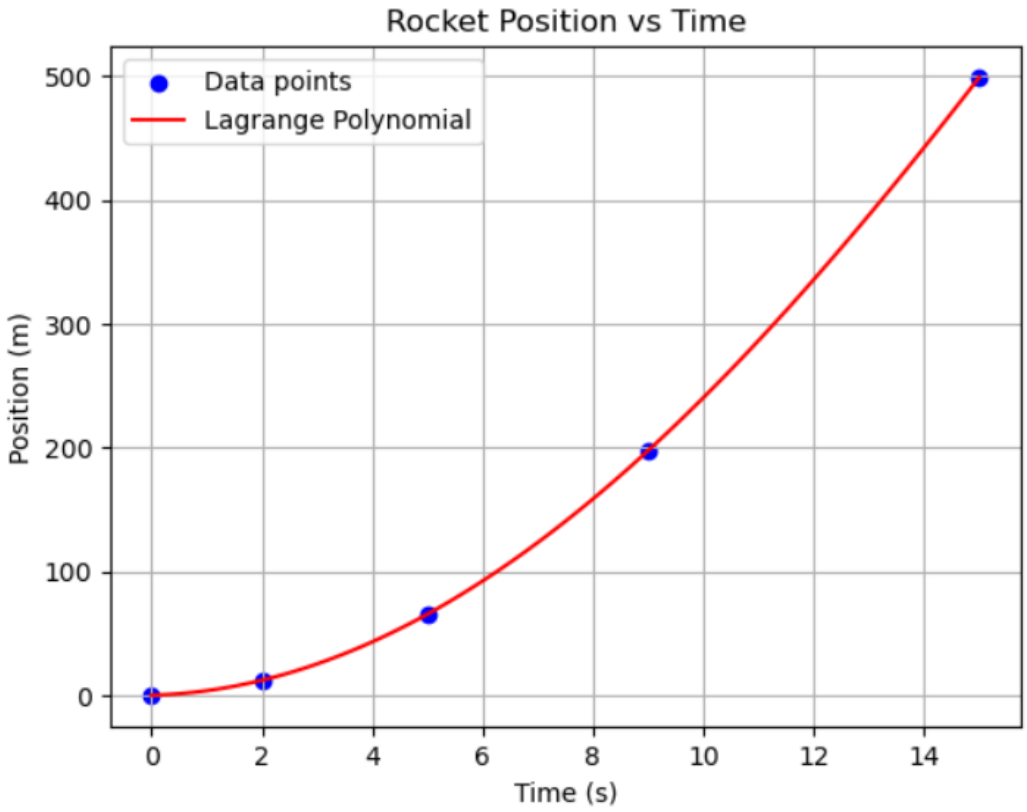
```
import numpy as np
from scipy.optimize import curve_fit

t = np.array([0, 10, 20, 30, 40, 50])
N = np.array([1000, 853, 727, 619, 527, 448])

def exp_decay(t, N0, lambda_):
    return N0 * np.exp(-lambda_ * t)

params, covariance = curve_fit(exp_decay, t, N, p0=[1000, 0.02])
N0_fit, lambda_fit = params

N_pred = exp_decay(t, N0_fit, lambda_fit)

residuals = N - N_pred
ss_res = np.sum(residuals**2)
ss_tot = np.sum((N - np.mean(N))**2)
r_squared = 1 - (ss_res / ss_tot)

print(f"R-squared: {r_squared:.4f}")
```

```
R-squared: 1.0000
```

2.

(a)

```
import numpy as np
from scipy.interpolate import lagrange
import matplotlib.pyplot as plt

t = np.array([0, 2, 5, 9, 15])
h = np.array([0.0, 12.1, 65.5, 197.6, 499.0])

poly = lagrange(t, h)

t_fit = np.linspace(0, 15, 100)
h_fit = poly(t_fit)

plt.scatter(t, h, color='blue', label='Data points')
plt.plot(t_fit, h_fit, color='red', label='Lagrange Polynomial')
plt.xlabel('Time (s)')
plt.ylabel('Position (m)')
plt.title('Rocket Position vs Time')
plt.legend()
plt.grid(True)
plt.show()

print("Lagrange polynomial coefficients:", poly.coef)
```




```
Lagrange polynomial coefficients: [-1.03276353e-03 -2.92022792e-03  2.41071937e+00  1.248504
27e+00
  0.00000000e+00]
```

(b)

```python
import numpy as np
from scipy.interpolate import lagrange
import matplotlib.pyplot as plt

t = np.array([0, 2, 5, 9, 15])
h = np.array([0.0, 12.1, 65.5, 197.6, 499.0])

poly = lagrange(t, h)

# Velocity v(t) = d/dt [h(t)]
velocity_poly = np.polyder(poly)

t_1 = 7
velocity_at_7 = velocity_poly(t_1)

print(f"\nVelocity at t = {t_1} seconds: {velocity_at_7:.4f} m/s")
```

```
Velocity at t = 7 seconds: 33.1524 m/s
```

3.

(a)

```
import numpy as np

t = np.array([0, 1, 2, 3, 4, 5])
x = np.array([2.0, 3.5, 5.0, 4.0, 2.0, 1.0])

velocity = []
for i in range(1, len(t) - 1):
    v = (x[i + 1] - x[i - 1]) / (2 * (t[i + 1] - t[i - 1]))
    velocity.append(v)

for i in range(len(velocity)):
    print(f"Velocity at t = {t[i + 1]} s: {velocity[i]:.2f} m/s")
```

```
Velocity at t = 1 s: 0.75 m/s
Velocity at t = 2 s: 0.12 m/s
Velocity at t = 3 s: -0.75 m/s
Velocity at t = 4 s: -0.75 m/s
```

(b)

```
import numpy as np

t = np.array([1, 2, 3, 4])
v = np.array([1.5, 0.25, -1.5, -1.5])

acceleration = []
for i in range(1, len(t) - 1):
    a = (v[i + 1] - v[i - 1]) / (2 * (t[i + 1] - t[i - 1]))
    acceleration.append(a)

for i in range(len(acceleration)):
    print(f"Acceleration at t = {t[i + 1]} s: {acceleration[i]:.2f} m/s^2")
```

```
Acceleration at t = 2 s: -0.75 m/s^2
Acceleration at t = 3 s: -0.44 m/s^2
```

4.

```python
import numpy as np

def f(x):
    return 2 * x**3 + 4 * x**2 - 6 * x + 1

a, b = 1, 5
h = 0.5
n = int((b - a) / h)
x = np.linspace(a, b, n + 1)
y = f(x)

integral_approx = (h / 2) * (f(a)+f(b) + 2 * np.sum(y[1:-1]))
print(f"Approximate integral (Trapezoidal Rule): {integral_approx:.4f}")

def F(x):
    return (x**4 / 2) + (4 * x**3 / 3) - 3 * x**2 + x

exact = F(b) - F(a)
print(f"Exact integral: {exact:.4f}")

error = abs(integral_approx - exact) / exact * 100
print(f"Percentage error: {error:.4f}%")
```

```
Approximate integral (Trapezoidal Rule): 413.0000
Exact integral: 409.3333
Percentage error: 0.8958%
```

5.

```python
import numpy as np

def f(x):
    return 4 * x**4 + 3 * x**3 - 2 * x**2 + 5 * x - 7

a, b = 0, 2
n = 4
h = (b - a) / n
x = np.linspace(a, b, n + 1)
y = f(x)

integral_approx = (h / 3) * (f(a) + 4 * np.sum(f(x[1:-1:2])) + 2*np.sum(f(x[2:-2:2]))+f(b))
print(f"Approximate integral (Simpson's 1/3 Rule): {integral_approx:.4f}")

def F(x):
    return (4 * x**5 / 5) + (3 * x**4 / 4) - (2 * x**3 / 3) + (5 * x**2 / 2) - 7 * x

exact = F(b) - F(a)
print(f"Exact integral: {exact:.4f}")

error = abs(integral_approx - exact) / exact * 100
print(f"Percentage error: {error:.4f}%")
```

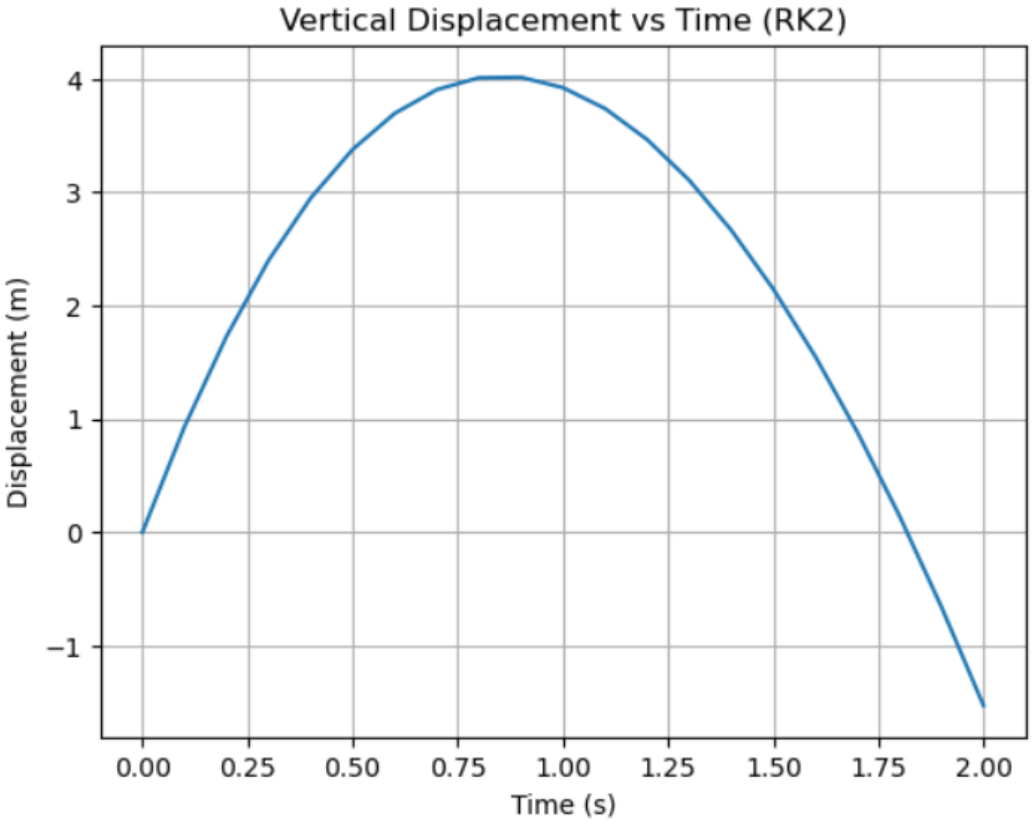
```
Approximate integral (Simpson's 1/3 Rule): 28.3333
Exact integral: 28.2667
Percentage error: 0.2358%
```

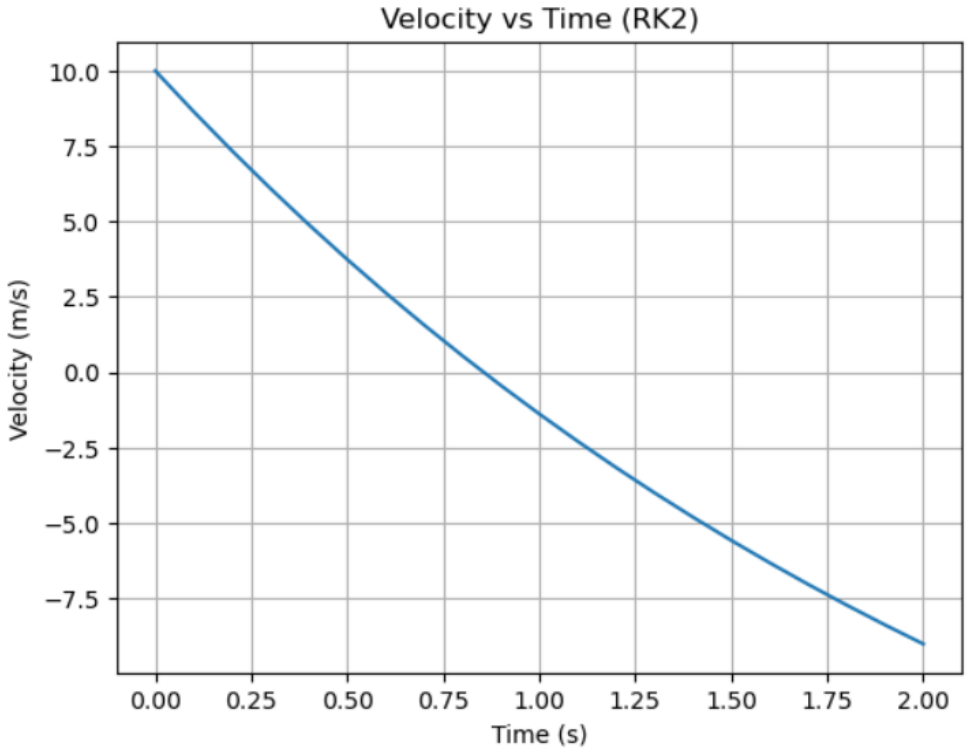
6.

(a)

```
import numpy as np
import matplotlib.pyplot as plt

m = 0.5
k = 0.2
g = 9.81

t0 = 0
total_time = 2
h = 0.1
t_values = np.arange(t0, total_time + h, h)

y0 = 0
v0 = 10

def system_dynamics(t, y, v):
    dy_dt = v
    dv_dt = (-k * v - m * g) / m
    return dy_dt, dv_dt

def runge_kutta_second_order(f, y0, v0, t_values, h):
    y_values = np.zeros_like(t_values)
    v_values = np.zeros_like(t_values)

    y_values[0] = y0
    v_values[0] = v0

    for i in range(len(t_values) - 1):
        t = t_values[i]
        y = y_values[i]
        v = v_values[i]
```

```
        k1_y, k1_v = f(t, y, v)
        k2_y, k2_v = f(t + h, y + h * k1_y, v + h * k1_v)

        y_values[i + 1] = y + h / 2 * (k1_y + k2_y)
        v_values[i + 1] = v + h / 2 * (k1_v + k2_v)

    return y_values, v_values

y_values, v_values = runge_kutta_second_order(system_dynamics, y0, v0, t_values, h)

plt.plot(t_values, y_values, label="RK2 Displacement")
plt.xlabel('Time (s)')
plt.ylabel('Displacement (m)')
plt.title('Vertical Displacement vs Time (RK2)')
plt.grid(True)
plt.show()

plt.plot(t_values, v_values, label="RK2 Velocity")
plt.xlabel('Time (s)')
plt.ylabel('Velocity (m/s)')
plt.title('Velocity vs Time (RK2)')
plt.grid(True)
plt.show()

print(f"RK method second order result at t = {final_time_RK:.2f} : {final_voltage_RK:.4f} m/s")
```



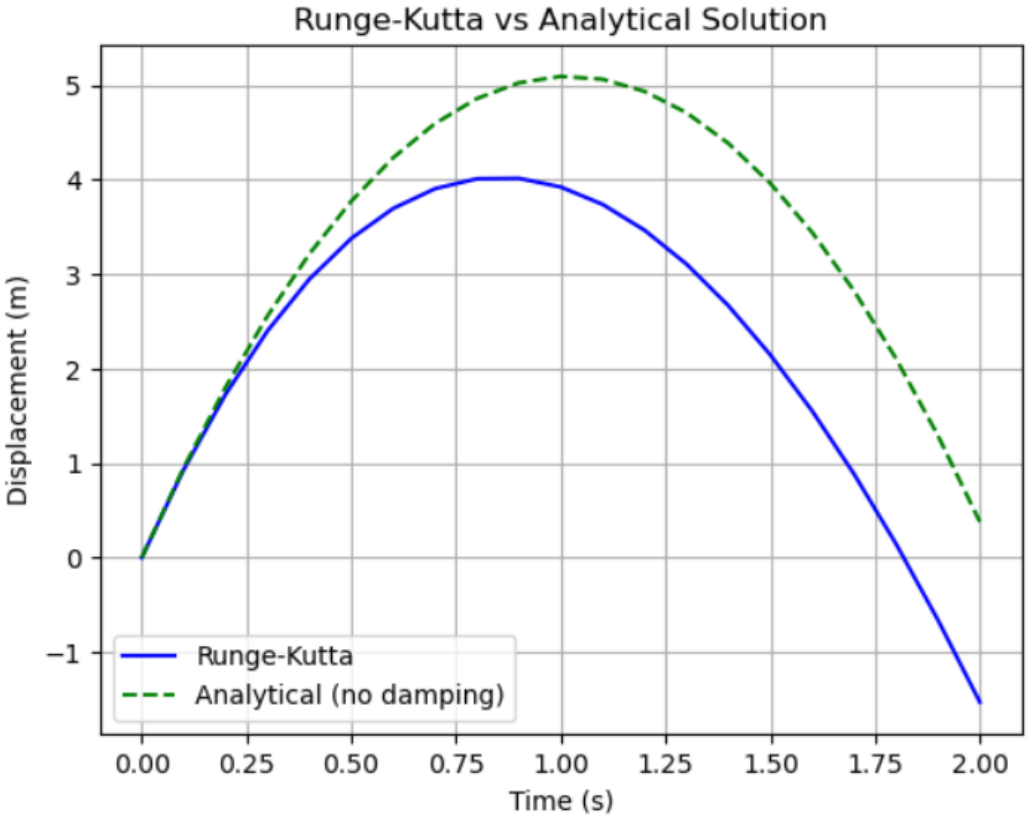



```
RK method second order result at t = 2.00 : -9.0119 m/s
```

(b)

```
import numpy as np
import matplotlib.pyplot as plt

m = 0.5
k = 0.2
g = 9.81

t0 = 0
total_time = 2
h = 0.1
t_values = np.arange(t0, total_time + h, h)

y0 = 0
v0 = 10

def system_dynamics(t, y, v):
    dy_dt = v
    dv_dt = (-k * v - m * g) / m
    return dy_dt, dv_dt

def runge_kutta_second_order(f, y0, v0, t_values, h):
    y_values = np.zeros_like(t_values)
    v_values = np.zeros_like(t_values)

    y_values[0] = y0
    v_values[0] = v0

    for i in range(len(t_values) - 1):
        t = t_values[i]
        y = y_values[i]
        v = v_values[i]
```

```
        k1_y, k1_v = f(t, y, v)
        k2_y, k2_v = f(t + h, y + h * k1_y, v + h * k1_v)

        y_values[i + 1] = y + h / 2 * (k1_y + k2_y)
        v_values[i + 1] = v + h / 2 * (k1_v + k2_v)

    return y_values, v_values

y_values, v_values = runge_kutta_second_order(system_dynamics, y0, v0, t_values, h)

y_analytical = -0.5 * g * t_values**2 + v0 * t_values

plt.plot(t_values, y_values, 'b-', label='Runge-Kutta')
plt.plot(t_values, y_analytical, 'g--', label='Analytical (no damping)')
plt.xlabel('Time (s)')
plt.ylabel('Displacement (m)')
plt.title('Runge-Kutta vs Analytical Solution')
plt.legend()
plt.grid(True)
plt.show()
```


Runge-Kutta vs Analytical Solution
Displacement (m)
Time (s)
Runge-Kutta
Analytical (no damping)
5
4
3
2
1
0
−1
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00